SKHU Agent 사용 설명서

> **SKHU Agent V1.0** — Jupyter Notebook 기반 RAG(Retrieval-Augmented Generation) 질의응답 데스크탑 앱

목차

1. 개요 및 실행
2. 화면 구성
3. ⚙ 설정 패널 (좌측)
4. 💬 채팅 탭
5. 📄 문서 탐색 탭
6. 🕸 그래프 탐색 탭
7. 📓 노트북 뷰어 탭
8. 📁 디렉토리 탭
9. 설정 파일 가이드
10. Force Mode 상세 가이드
11. 자주 묻는 질문 / 트러블슈팅

---

1. 개요 및 실행

SKHU Agent는 `work/` 폴더에 있는 **Jupyter Notebook(`.ipynb`) 파일만** 분석하여, **자연어 질문에 답변하는 RAG 기반 AI 어시스턴트**입니다.

> ⚠ **`.ipynb` 형식의 Jupyter Notebook 파일만 인식합니다.** `.py`, `.txt`, `.pdf`, `.csv` 등 다른 형식의 파일은 분석 대상에 포함되지 않습니다.

- **Vector RAG**, **BM25 키워드 검색**, **Graph RAG**를 결합한 하이브리드 검색 파이프라인을 사용합니다.
- 검색된 노트북 셀 내용을 LLM에 전달하여 한국어로 답변을 생성합니다.

실행 방법

배포 버전 (`.exe` 실행 파일)

1. `dist/SKHU_Agent/` 폴더 내 `env.txt` 파일을 열어 OpenAI API Key를 입력합니다.

   ```
   OPENAI_API_KEY=sk-...
   ```

2. `dist/SKHU_Agent/SKHU_Agent.exe`를 더블클릭하여 실행합니다.
3. 앱이 시작되면 좌측 설정 패널의 🚀 **RAG 시스템 구축** 버튼을 클릭합니다.
4. 구축 완료 후 채팅 탭에서 질문합니다.

> **분석 대상 노트북 변경**: `dist/SKHU_Agent/work/` 폴더 안의 `.ipynb` 파일을 교체한 뒤 🚀 **RAG 시스템 구축**을 다시 클릭하면 됩니다.

> **환경변수 설정 필수**: 실행 전 `env.txt`에 API Key를 설정해야 합니다. → 9. 설정 파일 가이드 참조

---

2. 화면 구성

```
┌──────────────────────────────────────────────────────────────┐
│ SKHU Agent V1.0                                              │
├──────────────────┬──────────────────────────────────────────┤
│                  │ 💬 채팅  📄 문서탐색  ⚙ 그래프  📓 노트북  📁 디렉토리 │
│ ⚙ 설정 패널      │                                          │
│                  │                                          │
│ • 노트북 경로    │          (탭 콘텐츠 영역)                 │
│ • LLM 설정       │                                          │
│ • Embedding      │                                          │
│ • 검색 모드      │                                          │
│ • Force 워커     │                                          │
│                  │                                          │
│ 🚀 RAG 구축      │                                          │
│                  │                                          │
│ 📊 통계          │                                          │
└──────────────────┴──────────────────────────────────────────┘
│ 상태바                                                       │
└──────────────────────────────────────────────────────────────┘
```

- **좌측**: 설정 패널 (항상 표시)
- **우측**: 5개 탭 (채팅, 문서 탐색, 그래프 탐색, 노트북 뷰어, 디렉토리)
- **하단 상태바**: RAG 구축 상태, 파일 변경 감지 알림 표시

---

3. ⚙ 설정 패널 (좌측)

📁 노트북 디렉토리

항목	설명
경로 입력	분석할 `.ipynb` 파일이 있는 폴더 (기본값: `work`)
… 버튼	폴더 탐색기로 선택

🤖 LLM 설정

항목	설명
Base URL	LLM API 엔드포인트. OpenAI 공식 서버는 비워둠. 로컬 서버(Ollama 등)는 `http://localhost:8000/v1` 형식으로 입력
모델명	사용할 LLM 모델명 (기본값: `gpt-4o-mini`)
API Key	OpenAI API Key (입력 시 마스킹됨)

🔢 Embedding 설정

항목	설명
Embedding Base URL	임베딩 전용 서버 URL (선택사항, 비워두면 LLM Base URL 사용)
Embedding 모델	임베딩 모델명 (기본값: `text-embedding-ada-002`)
OpenAI API Key	임베딩용 API Key

🔍 검색 모드

채팅 탭의 일반 RAG 질문에 사용할 검색 방식을 선택합니다.

옵션	설명
🔀 통합 (Vector + BM25 + Graph)	세 가지 검색 방식을 병합하여 사용 (기본값, 권장)
📐 Vector RAG만	FAISS 임베딩 기반 의미 검색만 사용
🔤 BM25만	키워드 빈도 기반 검색만 사용
🕸️ Graph RAG만	셀 관계 그래프 기반 검색만 사용

⚡ Force Mode — 병렬 워커 수

Force Mode(`/f`) 사용 시 동시에 실행할 LLM 호출 수를 설정합니다.

- 범위: **1 ~ 10** (기본값: `3`)
- 값이 클수록 빠르지만 API 사용량이 증가합니다.
- `env.txt`의 `FORCE_WORKERS` 값이 초기값으로 사용됩니다.

🚀 RAG 시스템 구축 버튼

처음 실행하거나 노트북 파일이 변경된 경우 반드시 클릭해야 합니다.

- 클릭 시: 노트북 파싱 → FAISS/BM25 인덱스 생성 → 그래프 구축
- 인덱스는 `.rag_cache/` 폴더에 캐시되어 다음 실행 시 재사용됩니다.
- 구축 완료 후 채팅 탭에 **예시 질문 칩**이 자동 생성됩니다.

> **파일 변경 감지**: 앱이 실행 중에도 30초마다 노트북 파일 변경을 감지하여 상태바에 알림을 표시합니다. 변경이 감지되면 RAG 재구축을 권장합니다.

📊 인덱스 통계

RAG 구축 완료 후 표시:

- 노트북 수, 총 셀 수, 코드 셀 수, 마크다운 셀 수
- 그래프 노드 수 · 엣지 수

---

4. 💬 채팅 탭

RAG 시스템을 활용한 질의응답 인터페이스입니다.

기본 사용 방법

1. 좌측 설정 패널에서 🚀 **RAG 시스템 구축** 버튼을 클릭하여 인덱스를 준비합니다.
2. 하단 입력창에 질문을 입력합니다.
3. **전송** 버튼을 클릭하거나 `Enter`를 눌러 전송합니다.
4. AI 답변이 스트리밍으로 표시됩니다.

예시 질문 칩

RAG 구축 완료 후, 노트북 내용을 분석하여 **추천 예시 질문**이 파란색 칩 형태로 자동 생성됩니다.

- 칩을 클릭하면 해당 질문이 입력창에 자동 입력 및 전송됩니다.
- 노트북 내용에 따라 질문이 달라집니다.

추천 쿼리 칩

AI 답변 완료 후 **후속으로 물어볼 만한 추천 검색어**가 자동으로 생성됩니다.

- 추천 칩을 클릭하면 바로 해당 쿼리로 질문합니다.

스트리밍 중지

AI가 답변을 생성하는 중에 **전송** 버튼이 ▣ **(빨간색)** 으로 바뀝니다.

- ▣ 버튼 클릭 시: 현재까지 생성된 부분 답변을 표시하고 즉시 중지합니다.
- 답변 완료 또는 중지 후 버튼이 **전송**으로 복원됩니다.

검색 결과 상세

답변 아래 🔍 **검색 결과 상세** 섹션을 클릭하면 펼칩니다.

- 어떤 노트북의 몇 번째 셀이 참조되었는지 확인할 수 있습니다.
- Vector RAG / BM25 / Graph RAG 각각의 검색 결과가 표시됩니다.

대화 초기화

입력창 우측 🗑 버튼 클릭 시 채팅 히스토리가 초기화됩니다.

Force Mode

`/f` 접두어를 사용한 전수 검색 모드입니다. → 10. Force Mode 상세 가이드 참조

---

5. 📄 문서 탐색 탭

RAG 인덱스에 포함된 **모든 셀을 직접 탐색**하는 탭입니다.

필터 및 검색

항목	설명
노트북 드롭다운	특정 노트북만 필터링 (기본: 전체 노트북)

항목	설명
타입 드롭다운	`code` / `markdown` / 전체 타입 필터
키워드 검색창	셀 내용에서 키워드로 검색 (실시간 필터링)

셀 목록

- 🟠 **코드 셀** (주황색)
- 🟣 **마크다운 셀** (보라색)
- 미리보기 텍스트 120자 표시
- 최대 50개 셀 표시 (필터링으로 범위를 좁혀 탐색 권장)

전체 내용 보기

- 셀을 **더블클릭**하면 팝업 창에서 전체 소스 코드/텍스트를 확인할 수 있습니다.

---

6. 🕸 그래프 탐색 탭

노트북 셀들 간의 **관계 그래프**를 탐색하는 탭입니다.

그래프 구조

Graph RAG는 두 종류의 엣지로 셀 관계를 표현합니다:

엣지 타입	설명	가중치 감쇠
`sequential`	같은 노트북 내 인접한 셀	0.5
`shared_var`	동일한 변수명을 공유하는 코드 셀	0.8

좌측 패널

- **통계 카드**: 전체 노드 수, 엣지 수, sequential/shared_var 엣지 수 표시
- **노드 검색**: 노트북명 또는 셀 번호로 노드를 검색
  - 검색 결과 클릭 시 해당 노드의 이웃 노드 표시
  - ← **이전 N개** (보라색): 해당 노드로 들어오는 선행 노드
  - → **다음 N개** (초록색): 해당 노드에서 나가는 후속 노드

우측 패널 — 엣지 목록

- 그래프의 엣지 최대 50개를 표로 표시
- 출발 노드, 도착 노드, 관계 타입, 공유 변수명 확인 가능

---

7. 📓 노트북 뷰어 탭

노트북 파일의 **셀 내용 확인 및 AI 요약** 기능을 제공합니다.

좌측 — 노트북 목록

- 체크박스로 노트북을 개별 선택하거나 **전체선택** 체크박스로 전체 선택/해제
- 파란색 표시: 요약이 완료된 노트북
- **요약 생성** 버튼: 선택한 노트북의 요약을 LLM으로 생성
  - 이미 요약된 노트북은 제외하고 `(N개)` 형태로 미요약 수를 표시
- **중지** 버튼: 요약 생성 중 즉시 중단

우측 — 콘텐츠 뷰

상단의 **셀 보기 / 요약 보기** 버튼으로 뷰를 전환합니다.

셀 보기

노트북 선택 시 해당 노트북의 모든 셀을 순서대로 표시합니다.

- **코드 셀** (주황색 테두리): 셀 번호와 코드 내용
- **마크다운 셀** (보라색 테두리): 셀 번호와 마크다운 텍스트

요약 보기

요약 생성 완료 후 **요약 보기** 버튼 클릭 시 전환됩니다.

- 각 노트북의 LLM 요약 카드가 마크다운 형식으로 렌더링됩니다.
- 요약은 디스크에 캐시되어 앱 재시작 후에도 유지됩니다.
  - 노트북 내용이 변경된 경우 자동으로 재요약합니다 (MD5 해시 비교).

---

8. 디렉토리 탭

설정된 **노트북 디렉토리의 파일 구조**를 트리 형태로 탐색합니다.

요약 정보

- 루트 경로, 노트북 수, 총 셀 수 표시

파일 트리

아이콘	색상	설명
	파란색	디렉토리
	초록색	Jupyter Notebook (`.ipynb`) — 셀 수 표시
	주황색	Python 파일 (`.py`)
	보라색	Markdown 파일 (`.md`)
	하늘색	CSV 파일 (`.csv`)
	노란색	JSON 파일 (`.json`)
	회색	텍스트 파일 (`.txt`)

- 파일 크기: B / KB / MB 단위로 표시

- 노트북: `N cells · size` 형태로 셀 수와 크기 동시 표시

---

9. 설정 파일 가이드

`env.txt` / `.env` — 환경변수 설정

앱 실행 전 필수 설정 파일입니다. `env_example.txt`를 복사하여 사용합니다.

> **배포 버전 경로**: `dist/SKHU_Agent/env.txt`

```
# LLM 설정
OPENAI_API_KEY=sk-...           # OpenAI API Key (필수)
LLM_MODEL=gpt-4o-mini           # 사용할 LLM 모델명
LLM_BASE_URL=                   # 로컬 서버 URL (OpenAI 공식은 빈값)

# Embedding 설정
EMBEDDING_BASE_URL=             # 임베딩 서버 URL (선택)
EMBEDDING_MODEL=text-embedding-ada-002

# Force Mode
FORCE_WORKERS=3                 # 병렬 워커 초기값 (1~10)
```

> `env_loader.py`가 `env.txt`를 자동으로 읽어 환경변수에 로드합니다.

`config.txt` — RAG 파이프라인 파라미터

프로젝트 루트에 `KEY=VALUE` 형식으로 작성합니다. 파일이 없으면 아래 기본값이 사용됩니다.

키	기본값	설명
`VECTOR_K`	`5`	Vector 검색 상위 K개 문서
`BM25_K`	`5`	BM25 검색 상위 K개 문서
`GRAPH_K`	`5`	Graph RAG 상위 K개 문서
`GRAPH_HOPS`	`2`	그래프 멀티홉 탐색 깊이
`SEQ_DECAY`	`0.5`	sequential 엣지 점수 감쇠율
`VAR_DECAY`	`0.8`	shared_var 엣지 점수 감쇠율
`KEYWORD_BOOST`	`0.4`	키워드 매칭 점수 부스트
`SEED_COUNT`	`3`	Graph RAG용 Vector 시드 문서 수
`VECTOR_WEIGHT`	`0.6`	앙상블 내 Vector 가중치
`BM25_WEIGHT`	`0.4`	앙상블 내 BM25 가중치
`MAX_DOCS`	`10`	LLM에 전달할 최대 문서 수

키	기본값	설명
`LLM_TEMPERATURE`	`0.2`	LLM 응답 다양성 (0.0~1.0)
`TRACE_DEBUG`	`false`	디버그 로그 활성화

TRACE_DEBUG 로그

`TRACE_DEBUG=true` 설정 시:

- `trace_logs/` 폴더에 쿼리별 `.txt` 파일 생성
- 파일명 예시: `20260330_143000_판다스_데이터프레임.txt`
- 내용: Vector RAG / BM25 / Graph RAG / Merged 4단계의 검색 결과 (노트북명, 셀 번호, 셀 내용)

`system_prompt.txt` — 일반 LLM 시스템 프롬프트

채팅 탭의 **일반 RAG 질문**에 사용할 시스템 프롬프트를 커스터마이징합니다.

- 파일이 없으면 기본 프롬프트 사용 (검색된 노트북 내용만 참조, 한국어 답변)
- 예시:

```
당신은 데이터 분석 교육 전문가입니다.
제공된 노트북 내용을 바탕으로 학생들이 이해하기 쉽게 설명해주세요.
반드시 한국어로 답변하세요.
```

`force_prompt.txt` — Force Mode 관련성 판단 프롬프트

`/f` Force Mode에서 각 청크의 **관련성을 판단**할 때 사용하는 시스템 프롬프트입니다.

- `system_prompt.txt`와 독립적으로 작동합니다.
- 파일이 없으면 기본 관련성 판단 프롬프트 사용

`summary_prompt.txt` — 노트북 요약 프롬프트

📓 **노트북 뷰어** 탭의 요약 생성 시 사용할 시스템 프롬프트입니다.

- 파일이 없으면 기본 프롬프트 사용

---

10. Force Mode 상세 가이드

Force Mode는 RAG 인덱스를 사용하지 않고 **모든 노트북 파일을 직접 전수 검사**하는 검색 모드입니다.

사용 방법

채팅 입력창에 `/f` 접두어를 붙여 질문합니다:

```
/f 판다스 데이터프레임 병합 방법
/f 결측값 처리 예제
/f 선형 회귀 모델 학습
```

> 전각 슬래시(/), 분수 슬래시(⁄) 등도 자동으로 `/f`로 변환됩니다.

동작 방식

1. 모든 `.ipynb` 파일을 **5셀 단위 청크**로 분할
2. N개의 병렬 워커가 동시에 각 청크에 대해 LLM으로 관련성 판단
3. 관련 있다고 판단된 청크의 내용을 채팅창에 **누적 표시**
4. 진행 상황이 프로그레스 바로 표시됨 (완료수/전체수)

Force Mode 진행 중 UI

```
Force Mode 검색 중… 12/48  [▓▓▓
```

2. 좌측 설정 패널에서 🚀 **RAG 시스템 구축** 버튼을 클릭했는지 확인
3. 하단 상태바의 오류 메시지 확인

Q. RAG 시스템 구축이 너무 오래 걸립니다.

A. 처음 실행 시 FAISS 임베딩 인덱스를 생성하므로 노트북 수에 따라 시간이 걸립니다. 두 번째 실행부터는 `.rag_cache/` 폴더의 캐시를 재사용하므로 빠릅니다.

Q. 노트북 파일을 수정했는데 채팅 답변이 이전 내용을 반영합니다.

A. 파일 변경 감지 알림이 상태바에 표시됩니다. 🚀 **RAG 시스템 구축** 버튼을 다시 클릭하여 인덱스를 재구축하세요.

Q. 로컬 LLM(Ollama 등)을 사용하려면 어떻게 설정하나요?

A. 설정 패널 **LLM Base URL** 항목에 로컬 서버 주소를 입력합니다:

```
http://localhost:11434/v1    # Ollama
http://localhost:8000/v1     # 기타 OpenAI 호환 서버
```

모델명도 로컬 모델명에 맞게 변경하세요 (예: `llama3.2`, `qwen2.5` 등).

Q. Force Mode가 RAG보다 좋은 건가요?

A. 상황에 따라 다릅니다. Force Mode는 전체 파일을 검사하므로 누락 없이 관련 내용을 찾을 수 있지만, LLM 호출 비용이 많이 발생합니다. 일반적인 질문에는 RAG를, 특정 코드나 예제를 반드시 찾아야 할 때는 Force Mode를 권장합니다.

Q. `TRACE_DEBUG=true` 로그를 어떻게 활용하나요?

A. `config.txt`에 `TRACE_DEBUG=true`를 추가하면, 각 질문마다 `trace_logs/` 폴더에 로그 파일이 생성됩니다. Vector RAG / BM25 / Graph RAG 각각이 어떤 셀을 검색했는지 확인하여 검색 품질 튜닝에 활용할 수 있습니다.

Q. 예시 질문 칩이 표시되지 않습니다.

A. 예시 질문 칩은 RAG 시스템 구축 완료 후 LLM이 노트북 내용을 분석하여 자동 생성합니다. RAG 구축이 완료되면 잠시 후 자동으로 표시됩니다.

SKHU Agent V1.0 — 박종범 강사 (jongbum3.park@sk.com)